

# LABORATOR 6

## Fire de executie

---

### 1. Scopul lucrării

Lucrarea isi propune studiul firelor de executie in Linux si realizarea de aplicatii care sa implementeze fire se executie.

### 2. Breviar teoretic

Intr-o lucrare de laborator anterioara au fost introduse elementele de baza referitoare la *proces*. Recapituland pe scurt, un proces era vazut ca fiind format dintr-o zona de cod, o zona de date, stiva si registri ai procesorului (Program Counter si altii). In consecinta, fiecare proces aparea ca o entitate distincta, independenta de celelalte procese aflate in executie la un moment dat. De asemenea, a fost remarcat faptul ca, avand in vedere ca procesorul poate rula la un moment dat un singur proces, procesele sunt executate pe rand, dupa un anumit algoritm de planificare, astfel incat, la nivelul aplicatiilor, acestea par ca se executa in paralel.

Se desprind, astfel, doua idei importante referitoare la procese:

- ruleaza independent, avand zone de cod, stiva si date distincte
- trebuie planificate la executie, astfel incat ele sa ruleze aparent in paralel

Executia planificata a proceselor presupune ca, la momente de timp determinate de algoritmul folosit, procesorul sa fie "luat" de la procesul care tocmai se executa si sa fie "dat" unui alt proces. Aceasta comutare intre procese (*process switching*) este o operatie consumatoare de timp, deoarece trebuie "comutate" toate resursele care apartin proceselor: trebuie salvati si restaurati toti registrii procesor, trebuie (re)mapate zonele de memorie care apartin de noul proces etc.

Un concept interesant care se regaseste in toate sistemele de operare moderne este acela de *fir de executie (thread)* in interiorul unui proces. Firele de executie sunt uneori numite *proces* *usoare (lightweight processes)*, sugerandu-se asemanarea lor cu procesele, dar si, intr-un anume sens, deosebirea dintre ele.

Un fir de executie trebuie vazut ca un flux de instructiuni care se executa *in interiorul unui proces*. Un proces poate sa fie format din mai multe asemenea fire, care se executa in paralel, *avand, insa, in comun toate resursele principale caracteristice procesului*. Prin urmare, in interiorul unui proces, firele de executie sunt entitati care ruleaza in paralel, impartind intre ele zona de date si executand portiuni distincte din acelasi cod. Deoarece zona de date este comuna, toate variabilele procesului vor fi vazute la fel de catre toate firele de executie, orice modificare facuta de catre un fir devenind vizibila pentru toate celelalte. Generalizand, un proces, asa cum era el perceptut in lucrarile de laborator precedente, este de fapt un proces format dintr-un singur fir de executie.

La nivelul sistemului de operare, executia in paralel a firelor de executie este obtinuta in mod asemanator cu cea a proceselor, realizandu-se o comutare intre fire, conform unui algoritm de planificare. Spre deosebire de cazul proceselor, insa, aici comutarea poate fi facuta mult mai rapid, deoarece informatiile memorate de catre sistem pentru fiecare fir de executie sunt mult mai putine decat in cazul proceselor, datorita faptului ca firele de executie au foarte putine resurse proprii. Practic, un fir de executie poate fi vazut ca un numarator de program,

o stiva si un set de registri, toate celelalte resurse (zona de date, identificatori de fisier etc) apartinand procesului in care ruleaza si fiind exploatate in comun.

## 1. Implementarea firelor de executie in Linux

Linux implementeaza firele de executie oferind, la nivel scazut, apelul sistem *clone()*:

```
pid_t clone(void *sp, unsigned long flags)
```

Funcția *clone()* este o interfata alternativa la funcția sistem *fork()*, ea avand ca efect crearea unui proces fiu, oferind, inasa, mai multe optiuni la creare.

Daca *sp* este diferit de zero, procesul fiu va folosi *sp* ca indicator al stivei sale, permitandu-se astfel programatorului sa aleaga stiva noului proces.

Argumentul *flags* este un sir de biti continand diferite optiuni pentru crearea procesului fiu. Octetul inferior din *flags* contine semnalul care va fi trimis la parinte in momentul terminarii fiului nou creat. Alte optiuni care pot fi introduse in cuvantul *flags* sunt: COPYVM si COPYFD. Daca este setat COPYVM, paginile de memorie al fiului vor fi copii fidele ale paginilor de memori ale parintelui, ca la funcția *fork()*. Daca COPYVM nu este setat, fiul va imparti cu parintele paginile de memorie ale acestuia. Cand COPYFD este setat, fiul va primi descriptorii de fisier ai parintelui ca si copii distincte, iar daca nu este setat, fiul va imparti descriptorii de fisier cu parintele.

Funcția returneaza PID-ul fiului in parinte si zero in fiu.

Prin urmare, apelul sistem *fork()* este echivalent cu:

```
clone(0, SIGCLD | COPYVM)
```

Se observa ca funcția *clone()* ofera suficiente facilitati pentru a putea crea primitive de tip fire de executie.

## 2. Utilizarea firelor de executie

In programe este indicat sa nu se foloseasca direct funcția *clone()*, in primul rand din cauza ca ea nu este portabila (fiind specifica Linux) si apoi pentru ca utilizarea ei este intrucativa greoaie.

Standardul POSIX 1003.1c, adoptat de catre IEEE ca parte a standardelor POSIX, defineste o interfata de programare pentru utilizarea firelor de executie, numita *pthread*. Interfata este implementata pe multe arhitecturi; mai mult, sistemele de operare care contineau biblioteci proprii de fire de executie (cum este SOLARIS) introduc suport pentru acest standard.

In Linux exista o biblioteca numita *LinuxThreads*, care implementeaza versiunea finala a standardului POSIX 1003.1c si utilizeaza funcția *clone()* ca instrument de creare a firelor de executie. In continuare, ne vom referi la aceasta biblioteca de functii si vom trece in revista o parte din primitivele introduse de catre ea.

### 2.1 Aspecte practice privind utilizarea bibliotecii LinuxThreads

Biblioteca *LinuxThreads* este disponibila automat in toate distributiile moderne de Linux (presupunand ca au fost instalate pachetele de development).

Pentru a putea utiliza fire de executie, este necesar ca programele sa fie compilate folosind comanda:

```
gcc -D_REENTRANT -lpthread <fisier.c> -o <executabil>
```

Se observa ca este necesara definirea constantei *\_REENTRANT* (din considerente legate de executia paralela a firelor) si ca trebuie inclusa explicit biblioteca *pthread* (numele sub care se regaseste *LinuxThreads*, conform POSIX).

### 2.2 Crearea firelor de executie

Un fir de executie se creaza folosind funcția

```
int pthread_create(pthread_t *thread, pthread_attr_t *attr,  
void *(*start_routine)(void *), void *arg);
```

Functia creeaza un *thread* care se va executa in paralel cu *thread*-ul creator. Noul fir de executie va fi format de functia *start\_routine* care trebuie definita in program avand un singur argument de tip **(void \*)**. Parametrul *arg* este argumentul care va fi transmis acestei functii. Parametrul *attr* este un cuvint care specifica diferite optiuni de creare a firului de executie. In mod obisnuit, acesta este dat ca NULL, acceptand optiunile implicite. Firul de executie creat va primi un identificator care va fi returnat in variabila indicata de parametrul *thread*. Functia returneaza 0 daca crearea a avut succes si un numar diferit de zero in caz contrar. *Thread*-ul va consta in executia functiei date ca argument, iar terminarea lui se va face ori apeland explicit functia *pthread\_exit()*, ori implicit, prin iesirea din functia *start\_routine*.

### **2.3 Terminarea firelor de executie**

Un fir de executie se poate termina apeland:

**void pthread\_exit(void \*retval) ;**

Valoarea *retval* este valoarea pe care *thread*-ul o returneaza la terminare. Starea returnata de firele de executie poate fi preluata de catre oricare din *thread*-urile aceluiasi proces, folosind functia:

**int pthread\_join(pthread\_t th, void \*\*thread\_return) ;**

Aceasta intrerupe firul de executie care o apeleaza pana cand firul de executie cu identificatorul *th* se termina, moment in care starea lui va fi returnata la adresa data de parametrul *thread\_return*.

Demn de observat este faptul ca *nu pentru toate firele de executie poate fi preluata starea de iesire*. De fapt, conform standardului POSIX, firele de executie se impart in doua categorii:

- **joinable** - ale caror stari pot fi preluate de catre celelalte fire din proces
- **detached** - ale caror stari nu pot fi preluate

In cazul *thread*-urilor *joinable*, in momentul terminarii acestora, resursele lor nu sunt complet dezalocate, asteptandu-se un viitor *pthread\_join* pentru ele. Firele de executie *detached* se dezaloca in intregime, starea lor devenind nedisponibila pentru alte fire de executie.

Tipul unui fir de executie poate fi specificat la crearea acestuia, folosind optiunile din argumentul *attr* (implicit este *joinable*). De asemenea, un fir de executie *joinable* poate fi "detasat" mai tarziu, folosind functia *pthread\_detach()*.

### **2.4 Observatii**

- Un proces, imediat ce a fost creat, este format dintr-un singur fir de executie, numit fir de executie principal (initial).
- Toate firele de executie din cadrul unui proces se vor executa in paralel.
- Datorita faptului ca impart aceeasi zona de date, firele de executie ale unui proces vor folosi in comun toate variabilele globale. De aceea, se recomanda ca in programe firele de executie sa utilizeze numai variabilele locale, definite in functiile care implementeaza firul, in afara de cazurile in care se doreste partajarea explicita a unor resurse.
- Daca un proces format din mai multe fire de executie se termina ca urmare a primirii unui semnal, toate firele de executie ale sale se vor termina.
- Daca un fir de executie apeleaza functia *exit()*, efectul va fi terminarea *intregului* proces, cu toate firele de executie din interior.
- Orice functie sau apel sistem care lucreaza cu sau afecteaza procese, va avea efect asupra intregului proces, indiferent de firul de executie in care a fost apelata functia respectiva. De exemplu,

functia *sleep( )* va "adormi" toate firele de executie din proces, inclusiv firul de executie principal (initial), indiferent de firul care a apelat-o.

- Standardul POSIX defineste si cateva primitive de sincronizare intre firele de executie pentru accesul la resursele comune, care nu fac obiectul lucrarii de fata.